

Day - 13: Timers & Counters

▼ Timer Working | Timer in Embedded System

1. Introduction to Timers

- **Definition:** A timer is a dedicated hardware peripheral located physically inside a microcontroller.
- **Primary Function:** It counts clock pulses provided by a crystal oscillator or peripheral clock to measure time, generate precise delays, and trigger regular events.
- **Independence:** The timer is an independent circuit with its own dedicated registers and functionalities. It operates without directly involving the main CPU.
- **Importance:** Timers are so essential for scheduling and synchronization that it is practically impossible to design a real-time embedded system without them.

2. Core Functions & Capabilities

Timers are highly versatile and can perform several critical tasks within an embedded system:

- **Counting:** Can be used as a counter to tally external events.
- **Delay Generation:** Generates 100% accurate time delays.
- **Task Scheduling:** Triggers different applications or functions at specific times.
- **Pulse Width Modulation (PWM):** Generates PWM signals to control the speed of motors or the intensity of lights (like car headlamps).
- **Event Capturing:** Records the exact time a specific external event occurred.

- **Interrupt Generation:** Creates interrupts based on time conditions to alert the processor.
- **Synchronization:** Synchronizes multiple different components or applications based on a unified time unit.

3. Working Mechanism & Block Diagram

The timer operates using a few sequential components:

1. **Clock Source:** Supplies the raw clock pulses, either from an external crystal oscillator or an internal peripheral clock.
2. **Prescaler:** An optional block used to "step down" or divide the incoming clock value. If stepping down is not needed, it is set to a value of 1.
3. **Timer Counter:** The main circuit that actively counts the clock pulses. While it can act as a down-counter, it is most commonly configured as an up-counter.
4. **Match Value / Register:** A predefined value set by the programmer. The timer constantly compares its current count to this match value.
5. **Timer Output Model:** When the timer reaches the match value (or overflows), it sends a signal to this model, which in turn generates the desired output, such as an interrupt or a PWM pulse.

4. Two Primary Operating Modes

The timer generally operates in one of two major modes:

- **Output Compare Mode:** The timer counts up and compares its value to a predefined "match" value. Once the values are equal, it generates an interrupt or triggers a specific functionality.
- **Input Capture Mode:** The timer runs continuously in a "free-running" state. When a sudden external event or interrupt occurs, the timer "captures" and records the exact time that the event happened so the system can evaluate it.

5. Additional Timer Configurations

Beyond the primary modes, timers can be configured in several specific ways based on application needs:

- **Simple Timer Mode:** The timer simply counts up to its maximum value and then rolls back to zero.
- **Auto Reload Mode:** Normally, a timer resets to 0 after completing a cycle. In auto-reload mode, it automatically resets to a custom, predefined value instead of zero.
- **Watchdog Timer Mode:** The timer acts as a safety mechanism to monitor the system and reset it if it crashes or hangs.
- **Timer Events Mode:** Triggers a sequence of distinct events at designated intervals (e.g., Event 1 at 10s, Event 2 at 20s).

6. Real-World Applications

Because of their ability to schedule and control time-based events, timers are heavily utilized in everyday electronics, including:

- Washing machines and microwaves (for cycling through different washing/heating stages).
- Traffic lights.
- Fitness equipment (like treadmills) and smartwatches.
- Alarm systems.

▼ Timer Programming | Timer Compare Mode

1. Overview of LPC 1768 Timers

- **Microcontroller Specs:** The LPC 1768 microcontroller features **four independent timers** (Timer 0, Timer 1, Timer 2, and Timer 3).
- **Architecture:** All four of these timers are **32-bit** in nature.
- **Operating Mode:** The notes specifically focus on the **Match/Compare mode**, where the timer continuously counts and compares its value to a predefined target to generate exact delays.

2. Key Timer Configuration Registers

To successfully write Embedded C code for these timers, developers must configure several specific registers:

- **TCR (Timer Control Register):** Used to control the fundamental operations of the timer. Bit 0 is used to enable or disable the timer, and Bit 1 is used to clear or reset the counters before starting.
- **PR (Prescaler Register):** Used to divide the main microcontroller clock by a specific value to slow down the timer count. For example, setting it to 23 divides the main clock by 24 ($23 + 1$).
- **TC (Timer Counter):** The actual core counter register that increments over time.
- **MR (Match Register / Compare Register):** Holds the specific "time-out" value the programmer wants to reach. The microcontroller has four match registers (MR0, MR1, MR2, and MR3).
- **MCR (Match Control Register):** Defines what action the timer should take once the timer counter (TC) successfully matches the value in the match register (MR).
- **IR (Interrupt Register):** A flag register that is set when a match condition occurs, allowing the system to trigger an interrupt.
- **CR (Capture Register):** Used specifically for capture mode operations (not utilized in this match/compare scope).

3. Configuring the Match Control Register (MCR)

The MCR allows you to trigger three distinct operations when a match occurs by setting specific bits:

- **Bit 0:** Generates an **interrupt**.
- **Bit 1:** Sets the timer to **repeat** (resetting and starting over automatically).
- **Bit 2:** Sets the timer to **stop** completely upon a match.
- *Application:* To create a continuous delay loop (like a 5-second repeating delay), you configure both Bit 0 and Bit 1 so the timer generates an interrupt and then repeats.

4. Step-by-Step Timer Configuration Code Logic

When setting up the timer in code, the configuration follows a specific sequence:

1. **Set the Prescaler:** Load a value into the `PR` register (e.g., `23`).
2. **Set the Match Value:** Load the desired delay value into `MR0`. For example, loading `5000000` (5 followed by six zeros) generates a 5-second delay.
3. **Configure Match Actions:** Set bits in the `MCR` to enable the interrupt and repeat mode.
4. **Reset Counters:** Use the `TCR` (bit 1) to reset the counter before starting.
5. **Enable Timer:** Finally, use the `TCR` (bit 0) to officially enable and start the timer.

5. Execution Flow & Interrupt Handling

- **The Match Condition:** In the main code's `while(1)` loop, the program continuously checks the Interrupt Register (`IR`) flag. This flag is only set when the `TC` value exactly matches the `MR` value.
- **Action Execution:** Once the flag is set (e.g., 5 seconds have passed), the code executes its intended action, such as printing "tim zero" on an LCD screen.
- **Clearing the Flag:** After executing the action, the programmer **must manually reset/clear the interrupt flag** in the `IR` register so the timer can properly detect the next match event.

6. Handling Multiple Timers Simultaneously

The architecture easily supports running multiple timers with different delays concurrently.

- **Setup:** You write two separate configuration functions (e.g., configuring Timer 0 with a 5-second delay, and Timer 1 with a 10-second delay).
- **Execution:** In the main loop, you add separate `if` conditions to check the individual interrupt flags for both Timer 0 and Timer 1.
- **Result:** Timer 0 will execute its action every 5 seconds, while Timer 1 executes its action independently every 10 seconds.

▼ Timer Capture Mode | How to Capture Timing

1. Introduction to Timer Capture Mode

- **Definition:** Capture mode is a timer configuration used to record the exact time stamp (timer unit) the moment a specific external event occurs (such as pressing or releasing a physical switch).
- **Purpose:** Unlike Compare Mode (which generates an event after a set time), Capture Mode listens for an external hardware event and instantly "captures" the exact time it happened.

2. Core Mechanism & Hardware Specifics

- **The Timer Counter (TC):** The timer runs continuously in the background like a free-running oscillator, constantly counting upwards.
- **The Capture Register (CR):** The microcontroller features dedicated Capture Registers (e.g., CR0, CR1, CR2, CR3). Whenever the targeted external event occurs, the current value of the Timer Counter (**TC**) is automatically copied by the hardware into one of these Capture Registers (**CR**).
- **LPC1768 Specifics:** In this specific microcontroller, Port Pin **P1.26** is utilized as the capture input, and it is designated to use **Capture Register 0 (CR0)**.

3. Key Registers Used

To program the capture mode, developers must configure several specific registers:

- **PINSEL3 (Pin Function Selection Register 3):** Used to configure standard GPIO port pin P1.26 specifically to function as a capture input.
- **PR (Prescaler Register):** Loaded with a value (e.g., 23) to divide the clock and define the timer's speed.
- **CCR (Capture Control Register):** A dedicated register used to configure the capture settings, such as enabling Capture 0.

- **TC (Timer Counter):** The active counter that increments over time.
- **CR0 (Capture Register 0):** The specific register where the captured time stamp is saved upon an event.

4. Step-by-Step Configuration & Coding Logic

Writing the code for Timer Capture Mode involves a straightforward setup process:

1. **Pin Configuration:** Use the `PINSEL3` register to change the function of pin P1.26 from a standard GPIO to a "Capture 0" input.
2. **Timer Configuration:**
 - Load the Prescaler Register (`PR`) with the desired step-down value (e.g., 23).
 - Configure the `CCR` (Capture Control Register) to enable the Capture 0 input. In this setup, it is configured to trigger on a **rising edge** (a transition from low to high).
 - Reset the timer and enable it so it starts running continuously.
3. **Main Execution Loop (while(1)):**
 - The hardware handles the actual capturing automatically. When an event occurs on P1.26, the `TC` value instantly drops into `CR0` without needing a software prompt.
 - Inside an infinite `while(1)` loop, the program continuously monitors `CR0` and simply copies its value into a custom global variable (e.g., `timer_capture_time_value`) so the developer can record and utilize the time stamp.

5. Execution Flow Example

- The timer is running continuously in the background.
- A user presses a button connected to pin P1.26, generating a rising edge signal (low to high).
- The microcontroller instantly recognizes this event and copies the current time from `TC` to `CR0` .

- The software reads **CR0** and saves the captured hexadecimal time stamp (e.g., **4FD9** or **1198CCB**) into the program's global variable for the developer to see.

▼ Timer Counter Mode | Event Counter | Timer vs Counter

1. Timer vs. Counter: The Fundamental Difference

- **Timer:** A timer specifically counts internal clock pulses provided by a crystal oscillator or peripheral clock. It uses these steady internal clock pulses to generate accurate delays or specific functionalities like PWM.
- **Counter:** A counter, on the other hand, is used to count **external events**.
- **The Key Distinction:** Timers count continuous internal clock pulses, whereas a timer configured in "counter mode" counts unpredictable external input events that arrive at a designated capture input pin.

2. Core Mechanism of Counter Mode

- In counter mode, the internal clock is not supplied to the prescaler; instead, external events (pulses) act as the source.
- These external events are fed directly into a specific capture input pin (such as port pin **P1.26**).
- Every time a valid pulse or event is received on this capture input, the main Timer Counter (TC) increments by exactly one.

3. The Crucial Register: CTCR (Count Control Register)

To switch the peripheral between acting as a standard timer or an event counter, developers must configure the **CTCR (Count Control Register)**. Bits 0 and 1 of this register dictate the mode:

- **00 (Timer Mode):** Operates as a regular timer, counting internal clock pulses continuously.
- **01 (Counter Mode - Rising Edge):** The counter increments only when it detects a rising edge (a low-to-high transition) on the capture input.

- **10 (Counter Mode - Falling Edge):** The counter increments only when it detects a falling edge (a high-to-low transition).
- **11 (Counter Mode - Both Edges):** The counter increments on both rising and falling edges of the input signal.

4. Configuration & Coding Logic

Setting up the timer in counter mode requires a few specific coding steps that closely resemble capture mode configuration:

1. **Pin Configuration:** Assign a dedicated pin (like P1.26) to act as the capture input for the external events.
2. **Disable Prescaler:** The prescaler register (`PR`) is made `0` (or bypassed entirely) because the counter relies purely on external events, making clock division unnecessary.
3. **Mode Selection:** Write to the `CTCR` register to select the desired counter mode (for example, setting it to `01` to capture events on a rising edge).

5. Execution Flow & Debugging

- **Timer Mode Behavior:** When configured normally (`CTCR = 00`), the Timer Counter (TC) is a free-running counter that increments continuously on its own without user input.
- **Counter Mode Behavior:** When configured in counter mode, the TC no longer runs continuously. Instead, it waits for an external trigger. If you physically simulate an event (like pressing and releasing pin P1.26 to create a rising edge), the system detects it and instantly increments the TC and the capture register by one.

▼ How to Measure Execution Time

1. Introduction to Execution Time Calculation

- **The Purpose:** Timers are essential for calculating exactly how much time an event or specific task takes to execute within an embedded system.
- **The Concept:** By capturing the timer's count before a task starts and immediately after it finishes, developers can determine the total execution time of that task.

2. The Free-Running Timer Configuration

- **Definition:** To accurately calculate execution time, the designated timer must be configured in **free-running mode**.
- **Mechanism:** A free-running timer operates continuously without any obstacles, interruptions, or specific conditions.
- **Configuration:** Unlike other timer modes, you do not set any match conditions or capture conditions; you simply turn the timer on and allow its counter to continuously run and increment.

3. The Core Logic (Pseudo Code)

Calculating the execution time follows a straightforward mathematical and logical sequence:

1. **Ensure Timer is Running:** Start the timer in the background in free-running mode.
2. **Capture Start Time:** Right as the microcontroller enters the function or event, capture the current value from the timer counter and save it as the `start_time`.
3. **Execute Task:** Allow the processor to fully execute the target task (e.g., performing calculations or printing a string).
4. **Capture End Time:** Before completing the task/function, capture the current value from the timer counter again and save it as the `end_time`.
5. **Calculate:** Apply the formula: **Execution Time = End Time - Start Time**.

4. Code Implementation Steps

When writing the actual Embedded C code, the execution flow generally looks like this:

- **Initialization:** First, configure the system clock, initialize the UART (so output can be printed to a terminal), and configure the timer.
- **Calling the Task:** Call the specific task function (e.g., `test_task`) that you want to measure.
- **Capturing Variables:** Inside the code, use dedicated variables to store the `start_time` and `end_time` fetched from the timer counter.

- **Printing Results:** Calculate the difference between the two variables, and use the UART to print the Start Time, End Time, and total Execution Time to the terminal for the developer to see.

5. Understanding the Output & Units

- **Microseconds (μs):** Because microcontrollers and timers typically run at high speeds (e.g., in Megahertz), the raw calculated difference from the timer counter represents the execution time in **microseconds**.
- **Conversion Example:** If the terminal displays a start time of `260,533` and an end time of `825,333`, subtracting them yields an execution time of `564,800`. This means the task took 564,800 microseconds to execute, which converts to exactly **0.564 seconds**.

▼ Interrupt Programming | ISR Configuration | Timer Interrupt

1. Introduction to Timer Interrupts

- **Concept:** While interrupts can be generated by various peripherals (like ADC or UART), this specific implementation focuses on generating interrupts using a **timer peripheral**.
- **Operating Mode:** To generate an interrupt, the timer must be properly configured into an **interrupt mode**, specifically utilizing the match/compare functionality.

2. Timer Hardware Configuration

The first step is setting up the timer's registers to dictate when the interrupt should occur:

- **Prescaler & Match Registers:** The Prescaler (`PR`) is configured to `23`, and Match Register 0 (`MR0`) is loaded with a value that generates a **5-second delay**.
- **Match Control:** The timer is set in compare/match mode with two specific bits enabled:
 - **Bit 0:** Configured to generate the actual interrupt when the timer reaches the match value.

- **Bit 1:** Configured for **repeat mode**, ensuring the timer resets and continues counting to generate the interrupt repeatedly every 5 seconds.
- **Activation:** Finally, the configuration resets the counter and enables the timer to start running.

3. NVIC Configuration (Interrupt Control)

Simply configuring the timer is not enough; the processor's interrupt controller (NVIC) must be instructed to listen for it using specific API functions:

- **NVIC_EnableIRQ:** This API function is used to explicitly enable the **Timer 0 interrupt** on the microcontroller.
- **NVIC_SetPriority:** This API function is used to assign a specific priority level to the interrupt. In this example, Timer 0 is assigned a priority of **1**.

4. The Interrupt Service Routine (ISR)

- **Definition:** An ISR (or Interrupt Handler) is a dedicated function that the processor automatically jumps to whenever the hardware generates an interrupt.
- **Execution Task:** In this specific ISR, the code simply prints the string **"timer zero IRQ"** to the terminal.
- **Crucial Step:** Before exiting the ISR, the code **must clear the interrupt flag**; otherwise, the system will not function correctly.

5. Main Execution Flow

- **Initialization:** The main code starts by initializing the basic system clocks, the timer configuration, and the UART (for printing to the terminal).
- **Main Loop:** The main code contains a simple, continuous loop that repeatedly prints the string **"normal"**, indicating that the CPU is operating in its normal condition.
- **The Interruption:** While the terminal continuously prints "normal", the timer runs in the background. **Every 5 seconds, the timer generates an interrupt**, causing the processor to temporarily pause the main loop.

- **Resuming:** The processor jumps to the ISR, prints "**timer 0 IRQ**", clears the flag, and then returns directly back to the main code to resume printing "normal" until the next 5 seconds elapse.